UNIVERSITY OF ENGINEERING & TECHNOLOGY MARDAN

COMPUTER SCIENCE

SUBJECT: ARTIFICIAL INTELLEGENCE

SUBMITTED BY & TO

NAME : RIFAQ AJMAL

REG NO : 23MDBCS 435

SECTION : A

SEMESTER : 4th

SUBMITTED T0 : MAM FAIZA

LAB # 2

TASK 2.1

Impement code for the game rock,paper scessor in python.

task 2.1.py - C:/Users/Rifaq Ajmal/Desktop/task 2.1.py (3.13.1)

File Edit Format Run Options Window Help

```python
import random

# Function to get the user's choice
def get_user_choice():
    user_choice = input("Enter rock, paper, or scissors: ").lower()
    return user_choice

# Function to get the computer's random choice
def get_computer_choice():
    choices = ['rock', 'paper', 'scissors']
    computer_choice = random.choice(choices)
    return computer_choice

# Function to decide the winner
def decide_winner(user, computer):
    if user == computer:
        return "It's a tie!"
    elif (user == 'rock' and computer == 'scissors') or \
         (user == 'scissors' and computer == 'paper') or \
         (user == 'paper' and computer == 'rock'):
        return "You win!"
    else:
        return "Computer wins!"

# Main game function
def play_game():
    user_choice = get_user_choice()
    computer_choice = get_computer_choice()

    print("You chose:", user_choice)
    print("Computer chose:", computer_choice)

    result = decide_winner(user_choice, computer_choice)
    print(result)

# Run the game
play_game()
```

OUTPUT

IDLE Shell 3.13.1

File Edit Shell Debug Options Window Help

```
Python 3.13.1 (tags/v3.13.1:0671451, Dec  3 2024, 19:06:28) [MSC v.1942 64 bit (AMD64)] on win32
Type "help", "copyright", "credits" or "license()" for more information.
>>>
=============== RESTART: C:/Users/Rifaq Ajmal/Desktop/task 2.1.py ==============
Enter rock, paper, or scissors: rock
You chose: rock
Computer chose: scissors
You win!
>>>
```

TASK 2.2

USING Strings:

Sort, Reverse,slices loops.

2.2.py - C:\Users\Rifaq Ajmal\Desktop\2.2.py (3.13.1)

File Edit Format Run Options Window Help

```python
# String manipulation in Python

# 1. Sorting a string
def sort_string(s):
    return ''.join(sorted(s))  # sorted() returns a list, so we join it back to a string

# 2. Reversing a string
def reverse_string(s):
    return s[::-1]

# 3. Slicing a string
def slice_string(s, start, end):
    return s[start:end]  # slice from index 'start' to 'end-1'

# 4. Loop through a string
def loop_string(s):
    for char in s:
        print(char)

# Test the functions
my_string = "python"

# Sorting the string
sorted_string = sort_string(my_string)
print("Sorted string:", sorted_string)

# Reversing the string
reversed_string = reverse_string(my_string)
print("Reversed string:", reversed_string)

# Slicing the string (e.g., slice from index 1 to 4)
sliced_string = slice_string(my_string, 1, 4)
print("Sliced string:", sliced_string)

# Loop through the string
print("Looping through the string:")
loop_string(my_string)
```

OUTPUT

IDLE Shell 3.13.1

File Edit Shell Debug Options Window Help

```
Python 3.13.1 (tags/v3.13.1:0671451, Dec  3 2024, 19:06:28) [MSC v.1942 64 bit (AMD64)] on win32
Type "help", "copyright", "credits" or "license()" for more information.
>>>
================= RESTART: C:\Users\Rifaq Ajmal\Desktop\2.2.py =================
Sorted string: hnopty
Reversed string: nohtyp
Sliced string: yth
Looping through the string:
p
y
t
h
o
n
>>>
```

TASK 2.3

Using Lists /2 dim List.

Create Sort, Extend, Append and Delete.

2.3.py - C:\Users\Rifaq Ajmal\Desktop\2.3.py (3.13.1)

File Edit Format Run Options Window Help

```python
# 1. Creating a list
my_list = [5, 2, 9, 1, 5, 6]

# 2. Creating a 2-dimensional list (list of lists)
my_2d_list = [
    [5, 2, 9],
    [1, 5, 6],
    [8, 3, 7]
]

# 3. Sorting a list
def sort_list(lst):
    return sorted(lst)

# Sorting a 2D list by sorting each row
def sort_2d_list(lst):
    return [sorted(row) for row in lst]

# 4. Extending a list (adding multiple elements to a list)
def extend_list(lst, new_elements):
    lst.extend(new_elements)  # modifies the list in place
```

```python

# 5. Appending an element to a list
def append_to_list(lst, new_element):
    lst.append(new_element)  # adds a single element at the end

# 6. Appending a new row to a 2D list
def append_to_2d_list(lst, new_row):
    lst.append(new_row)

# Test the operations
print("Original List:", my_list)
print("Original 2D List:", my_2d_list)

# Sorting
sorted_list = sort_list(my_list)
sorted_2d_list = sort_2d_list(my_2d_list)
print("\nSorted List:", sorted_list)
print("Sorted 2D List:", sorted_2d_list)
```

```python
# Extending the list
extend_list(my_list, [10, 11])
print("\nList after extend:", my_list)

# Appending to the list
append_to_list(my_list, 12)
print("List after append:", my_list)

# Appending to 2D list
append_to_2d_list(my_2d_list, [9, 4, 6])
print("2D List after appending a new row:", my_2d_list)

# 1. Deleting an element from a list by index using del
def delete_by_index(lst, index):
    del lst[index]  # removes the element at the specified index

# 2. Deleting an element from a list by value using remove
def delete_by_value(lst, value):
    lst.remove(value)  # removes the first occurrence of the specified value

# 3. Deleting an element from a list using pop (and returning it)
def pop_element(lst, index=None):
    if index is not None:
        return lst.pop(index)  # removes and returns the element at the specified inde
    return lst.pop()  # removes and returns the last element if no index is given

# 4. Deleting an entire row from a 2D list
def delete_row_2d_list(lst, row_index):
    del lst[row_index]  # deletes the row at the specified index

```

```python
# Test delete operations
my_list = [5, 2, 9, 1, 5, 6]
my_2d_list = [
    [5, 2, 9],
    [1, 5, 6],
    [8, 3, 7]
]

print("Original List:", my_list)
print("Original 2D List:", my_2d_list)

# 1. Delete an element by index
delete_by_index(my_list, 2)  # deletes element at index 2 (which is 9)
print("\nList after deleting by index (2):", my_list)

# 2. Delete an element by value
delete_by_value(my_list, 5)  # removes the first occurrence of 5
print("List after deleting by value (5):", my_list)

# 3. Pop an element (returns the popped element)
popped_element = pop_element(my_list, 1)  # removes and returns element at index 1
print("Popped element:", popped_element)
print("List after popping element at index 1:", my_list)

# 4. Delete a row from a 2D list
delete_row_2d_list(my_2d_list, 1)  # deletes the row at
```

OUTPUT

IDLE Shell 3.13.1

File Edit Shell Debug Options Window Help

```
Python 3.13.1 (tags/v3.13.1:0671451, Dec  3 2024, 19:06:28) [MSC v.1942 64 bit (AMD64)] on win32
Type "help", "copyright", "credits" or "license()" for more information.
>>>
================= RESTART: C:\Users\Rifaq Ajmal\Desktop\2.3.py =================
Original List: [5, 2, 9, 1, 5, 6]
Original 2D List: [[5, 2, 9], [1, 5, 6], [8, 3, 7]]

Sorted List: [1, 2, 5, 5, 6, 9]
Sorted 2D List: [[2, 5, 9], [1, 5, 6], [3, 7, 8]]

List after extend: [5, 2, 9, 1, 5, 6, 10, 11]
List after append: [5, 2, 9, 1, 5, 6, 10, 11, 12]
2D List after appending a new row: [[5, 2, 9], [1, 5, 6], [8, 3, 7], [9, 4, 6]]
Original List: [5, 2, 9, 1, 5, 6]
Original 2D List: [[5, 2, 9], [1, 5, 6], [8, 3, 7]]

List after deleting by index (2): [5, 2, 1, 5, 6]
List after deleting by value (5): [2, 1, 5, 6]
Popped element: 1
List after popping element at index 1: [2, 5, 6]
>>>
```

TASK 2.4

List Comprehension

2.4.py - C:\Users\Rifaq Ajmal\Desktop\2.4.py (3.13.1)

File Edit Format Run Options Window Help

```python
# 1. Basic List Comprehension: Squares of numbers from 0 to 9
squares = [x**2 for x in range(10)]
print("Squares:", squares)

# 2. List Comprehension with a Condition (Filtering): Even numbers from 0 to 9
evens = [x for x in range(10) if x % 2 == 0]
print("Even numbers:", evens)

# 3. Nested List Comprehension (2D Lists): Create a 3x3 matrix
matrix = [[i + j for j in range(3)] for i in range(3)]
print("2D List (Matrix):", matrix)

# 4. Flattening a 2D List: Flatten the matrix into a 1D list
flattened = [num for row in matrix for num in row]
print("Flattened 2D List:", flattened)

# 5. List Comprehension with Multiple Conditions: Squares of even numbers greater than 0
even_squares = [x**2 for x in range(10) if x % 2 == 0 and x > 0]
print("Even squares:", even_squares)

# 6. Applying a Function in List Comprehension: Using a function to square numbers
def square(x):
    return x * x

squared_list = [square(x) for x in range(10)]
print("Squared using function:", squared_list)

# 7. List Comprehension with Strings: Create a list of characters from a string
chars = [char for char in "hello"]
print("List of characters:", chars)

# 8. Using if-else in List Comprehension: Replace odd numbers with -1
modified_list = [x if x % 2 == 0 else -1 for x in range(10)]
print("Modified list (even stays, odd becomes -1):", modified_list)

# 9. List Comprehension with Multiple Loops: Cartesian product of two lists
cartesian_product = [(x, y) for x in [1, 2, 3] for y in ['a', 'b', 'c']]
print("Cartesian product:", cartesian_product)
```

OUTPUT

IDLE Shell 3.13.1

Edit Shell Debug Options Window Help

```
Python 3.13.1 (tags/v3.13.1:0671451, Dec  3 2024, 19:06:28) [MSC v.1942 64 bit (AMD64)] on win32
Type "help", "copyright", "credits" or "license()" for more information.
>
================= RESTART: C:\Users\Rifaq Ajmal\Desktop\2.4.py =================
Squares: [0, 1, 4, 9, 16, 25, 36, 49, 64, 81]
Even numbers: [0, 2, 4, 6, 8]
2D List (Matrix): [[0, 1, 2], [1, 2, 3], [2, 3, 4]]
Flattened 2D List: [0, 1, 2, 1, 2, 3, 2, 3, 4]
Even squares: [4, 16, 36, 64]
Squared using function: [0, 1, 4, 9, 16, 25, 36, 49, 64, 81]
List of characters: ['h', 'e', 'l', 'l', 'o']
Modified list (even stays, odd becomes -1): [0, -1, 2, -1, 4, -1, 6, -1, 8, -1]
Cartesian product: [(1, 'a'), (1, 'b'), (1, 'c'), (2, 'a'), (2, 'b'), (2, 'c'), (3, 'a'), (3, 'b'), (3, 'c')]
>
```

TASK 2.5

Random Module

Rand int, choice, shuffle, sample

2.5.py - C:\Users\Rifaq Ajmal\Desktop\2.5.py (3.13.1)

File Edit Format Run Options Window Help

```python
import random

# 1. Generating a random integer between two numbers using randint
random_int = random.randint(1, 100)
print("Random Integer between 1 and 100:", random_int)

# 2. Shuffling a list of numbers using shuffle
my_list = [1, 2, 3, 4, 5, 6, 7, 8, 9, 10]
random.shuffle(my_list)
print("Shuffled List:", my_list)

# 3. Getting a random sample from a list using sample
sample_list = random.sample(my_list, 3)  # get 3 random elements from the list
print("Random Sample of 3 elements:", sample_list)
```

OUTPUT

IDLE Shell 3.13.1

File Edit Shell Debug Options Window Help

```
Python 3.13.1 (tags/v3.13.1:0671451, Dec  3 2024, 19:06:28) [MSC v.1942 64 bit (AMD64)] on win32
Type "help", "copyright", "credits" or "license()" for more information.
>>>
================= RESTART: C:\Users\Rifaq Ajmal\Desktop\2.5.py =================
Random Integer between 1 and 100: 2
Shuffled List: [1, 5, 3, 6, 10, 4, 2, 8, 9, 7]
Random Sample of 3 elements: [7, 10, 9]
>>>
```

TASK 2.6

Punctuation Module

Replacement.

2.6.py - C:\Users\Rifaq Ajmal\Desktop\2.6.py (3.13.1)

File Edit Format Run Options Window Help

```python
import string

# 1. Function to replace punctuation with a specified character
def replace_punctuation(text, replacement_char=" "):
    return ''.join(replacement_char if char in string.punctuation else char for char in text

# Example sentence with punctuation
sentence = "Hello, world! How's it going? (Are you coding... or debugging?)"

# Replacing punctuation with a space
replaced_sentence = replace_punctuation(sentence, " ")
print("Sentence with punctuation replaced by spaces:", replaced_sentence)

# Replacing punctuation with an underscore
replaced_sentence_underscore = replace_punctuation(sentence, "_")
print("Sentence with punctuation replaced by underscores:", replaced_sentence_underscore)
```

OUTPUT

)LE Shell 3.13.1

Edit Shell Debug Options Window Help

```
Python 3.13.1 (tags/v3.13.1:0671451, Dec  3 2024, 19:06:28) [MSC v.1942 64 bit (AMD64)] on win32
Type "help", "copyright", "credits" or "license()" for more information.

================= RESTART: C:\Users\Rifaq Ajmal\Desktop\2.6.py =================
Sentence with punctuation replaced by spaces: Hello  world  How s it going   Are you coding    or debugging
Sentence with punctuation replaced by underscores: Hello_ world_ How_s it going_ _Are you coding___ or debugging_
```

THE END